

BiblioFlow

Projet DevOps - Application distribuée de gestion de bibliothèque

Auteur	Issa Kane
Formation	Projet DevOps
Version	1.0.0
Date	27 juin 2026

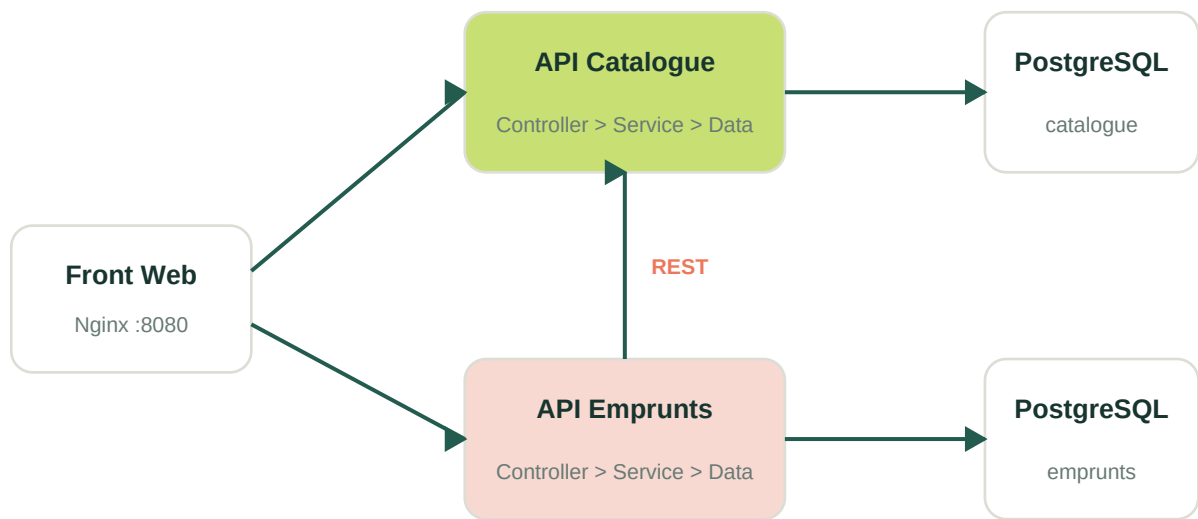
Dépôt Git - Pipeline CI - Architecture en couches - Deux backends Docker - Tests - Couverture - Qualité

1. Synthèse du projet

BiblioFlow répond au cahier des charges avec deux microservices Node.js indépendants. Le Catalogue gère les livres et leur stock. Le service Emprunts gère le cycle de vie des prêts et appelle le Catalogue par HTTP. Une interface web Nginx expose les deux API sous une origine unique.

Exigence	Preuve dans le projet
Dépôt et CI	GitHub et .github/workflows/ci.yml
Architecture en couches	data / services / controllers dans chaque API
Deux services back	catalog-api et loan-api dans Docker Compose
Tests et mocks web	56 tests Vitest, Supertest et mock du client HTTP
Couverture	99.40 % des lignes
Qualité	ESLint: 0 erreur, 0 avertissement
Bonus	Front web responsive et deux bases PostgreSQL

2. Architecture logicielle



Architecture interne de chaque backend

Controller : routes Express, validation Zod et codes HTTP. Service : règles métier, orchestration inter-service et compensation. Data : SQL paramétré et accès exclusif à PostgreSQL.

Le service Emprunts ne lit jamais la base Catalogue. Cette frontière rend les services déployables et testables séparément. Une réservation de stock est compensée si l'écriture de l'emprunt échoue ; le retour applique la stratégie inverse.

3. Conteneurisation et exploitation

Docker Compose orchestre cinq conteneurs. Les bases possèdent des volumes distincts et des healthchecks PostgreSQL. Les API attendent une base saine ; l'API Emprunts attend aussi le Catalogue. Nginx attend les deux API et publie le port 8080.

Conteneur	Rôle	Dépendance
web	Interface et reverse proxy	catalog-api, loan-api
catalog-api	Catalogue REST	catalog-db
loan-api	Emprunts REST	loan-db, catalog-api
catalog-db	Données des livres	-
loan-db	Données des emprunts	-

Validation Docker Compose : **OK**. Les images API utilisent Node.js 22 Alpine et s'exécutent avec l'utilisateur non-root node.

4. Tests automatisés

Le rapport JUnit contient **56 tests**, 0 échec et 0 erreur. Les tests couvrent toutes les couches et sont exécutés en moins d'une seconde sur la machine de génération.

Couche	Type de vérification
Data	Requêtes SQL, paramètres, valeurs absentes et résultats
Services	Règles métier, erreurs, concurrence et compensations
Controllers	Routes réelles avec Supertest, validation et erreurs HTTP
Inter-service	Client Catalogue avec réponses HTTP et pannes réseau simulées
Front web	DOM jsdom, confirmations et état de connexion
Configuration	Ports, variables obligatoires et pool PostgreSQL

Les controllers sont testés comme de vraies applications HTTP sans ouvrir de port. Le client inter-service accepte une implémentation fetch injectée : les tests simulent succès, erreur JSON, réponse invalide et panne réseau sans dépendance externe.

5. Couverture du code

La CI bloque la fusion si un seuil global n'est pas atteint. Les fichiers de démarrage, qui ne contiennent que le câblage et la gestion des signaux, sont exclus ; toutes les règles métier et couches applicatives restent mesurées.

Mesure	Résultat	Seuil CI	État
Instructions	98.84 %	>= 90 %	OK
Branches	98.27 %	>= 85 %	OK
Fonctions	96.22 %	>= 90 %	OK
Lignes	99.40 %	>= 90 %	OK

Le détail navigable est produit dans `coverage/index.html` et le résumé machine dans `coverage/coverage-summary.json`.

6. Qualité logicielle et CI

Contrôle	Résultat
ESLint	0 erreur / 0 avertissement
Prettier	Format vérifié dans la CI
npm audit	0 vulnérabilité déclarée
Tests et couverture	Seuils bloquants
Docker Compose	Configuration valide ; build contrôlé en CI
Test E2E Docker	Validé localement et en CI
Dependabot	npm, GitHub Actions et Docker surveillés

Le workflow GitHub Actions exécute deux jobs parallèles : Qualité et tests d'une part, construction Docker et parcours E2E d'autre part. Les permissions sont limitées à la lecture du dépôt et les jobs ont un délai maximal.

Mesures complémentaires : validation stricte des entrées, SQL paramétré, stock non négatif garanti par SQL, limite de corps JSON, en-têtes Helmet, politique CSP Nginx et messages d'erreur internes masqués.

7. Utilisation

Démarrage complet : `docker compose up --build`, puis ouverture de `http://localhost:8080`.
Les données de démonstration sont créées de façon idempotente au premier démarrage.

Méthode	Route	Usage
GET	/health (:3001/:3002)	Santé des API
GET	/api/books	Catalogue
POST	/api/books	Nouveau livre
PATCH	/api/books/:id/stock	Stock atomique
GET	/api/loans	Emprunts
POST	/api/loans	Nouvel emprunt
POST	/api/loans/:id/return	Retour

La documentation contractuelle complète se trouve dans `docs/openapi.yaml`. Le README décrit le démarrage local, les commandes de qualité et les décisions de fiabilité.

8. Preuve Google Labs

Capture fournie par l'étudiant et intégrée sans altération.

Progress

Track your learning activities, pick up where you left off, and celebrate what you've completed

Course

Lab

Quiz

Game

Learning path

Classroom

Show All

Activity	Type	Date started	Date finished	Score	Passed
Implementing Cloud Load Balancing for Compute Engine	Course	May 28, 2026	May 28, 2026	Assessment: 100%	✓
A Tour of Google Cloud Hands-on Labs	Lab	May 28, 2026	May 28, 2026	Assessment: 100%	✓
Getting Started with Google Cloud	Path	May 28, 2026	May 28, 2026	Assessment: 100%	✓
Infrastructure as Code with Terraform	Course	May 28, 2026	May 28, 2026	Assessment: 100%	✓